

WebApps i aplikacje mobilne

Projektowanie i programowanie aplikacji PWA i mobilnych cross-platform

Projekt

Informacje ogólne

Przedmiot: Projektowanie i programowanie aplikacji PWA i mobilnych cross-platform

Forma zaliczenia: Projekt semestralny (praca w zespołach 1–3 osobowych)

Wymiar godzinowy: 24 godziny lekcyjne (8 bloków × 3h)

Termin oddania: Ostatnie zajęcia w semestrze

Cel projektu

Celem projektu jest zaprojektowanie, zaimplementowanie i wdrożenie kompletnego systemu składającego się z:

- **Backendu z REST API** – serwer udostępniający dane dla obu klientów
- **Aplikacji PWA** – dostępnej przez przeglądarkę, działającej offline
- **Aplikacji mobilnej cross-platform** – działającej na Android (i opcjonalnie iOS)

Wszystkie trzy warstwy komunikują się przez wspólne API i prezentują spójny interfejs graficzny. Projekt ma na celu praktyczne zastosowanie wiedzy z całego cyklu zajęć, od analizy wymagań po wdrożenie produkcyjne.

Wymagania projektowe

Wybór technologii

Zespół może wybrać dowolną kombinację technologii, która spełni wymagania projektu. Poniżej przedstawiono rekomendowany stos oraz alternatywy:

Warstwa	Rekomendowany stos	Alternatywy
Backend + API	Django + Django REST Framework (Python)	FastAPI, Express.js (Node.js), ASP.NET Core, Spring Boot
Baza danych	PostgreSQL	MySQL, SQLite (dev), MongoDB
PWA (frontend)	React + Vite + Service Worker	Vue.js, Angular, Svelte, vanilla JS
Aplikacja mobilna	React Native (Expo)	Flutter (Dart), Kotlin Multiplatform, .NET MAUI, Ionic
Wdrożenie backend	Railway / Render	Heroku, VPS (DigitalOcean), Docker + dowolny hosting
Wdrożenie PWA	Vercel / Netlify	GitHub Pages, Cloudflare Pages, Firebase Hosting

Wybór technologii należy uzasadnić w dokumentacji projektu.

Wymagania funkcjonalne

Backend i API (wymagane):

- **REST API** – obsługa operacji CRUD dla głównych zasobów aplikacji
- **Baza danych** – relacyjna lub dokumentowa, z poprawnym modelem danych
- **Uwierzytelnianie** – rejestracja, logowanie, zabezpieczenie endpointów (JWT/token)

Aplikacja PWA (wymagane):

- **Manifest i Service Worker** – instalowalność, cache'owanie zasobów
- **Tryb offline** – podstawowa funkcjonalność bez połączenia z internetem
- **Responsywność** – poprawne działanie na desktop i mobile
- **Minimum 3 widoki/strony** z nawigacją

Aplikacja mobilna (wymagane):

- **Minimum 3 ekrany** z nawigacją
- **Komunikacja z API** – te same endpointy co PWA
- **Lokalne przechowywanie danych** (cache, AsyncStorage, SQLite itp.)
- **Minimum 1 natywna funkcja** – kamera, geolokalizacja, powiadomienia push, biometria lub czujniki

Wspólne wymagania:

- **Spójny interfejs** – ten sam design system (kolory, typografia, komponenty) w PWA i apce mobilnej
- **Synchronizacja danych** – zmiany w jednym kliencie widoczne w drugim po synchronizacji
- **System logowania** – to samo konto działa w PWA i apce mobilnej

Wymagania niefunkcjonalne

- **Bezpieczeństwo** – HTTPS, bezpieczne tokeny, walidacja danych, ochrona przed XSS/CSRF
- **Stabilność** – aplikacje nie powinny się zawieszać przy typowym użytkowaniu
- **Jakość kodu** – czytelna struktura, komentarze, sensowne nazewnictwo
- **Wersjonowanie** – regularne commity w Git z opisowymi komunikatami
- **Dostępność** – czytelne czcionki, odpowiedni kontrast, obsługa różnych rozmiarów ekranu

Etapy pracy nad projektem

Projekt jest realizowany w 8 blokach po 3 godziny lekcyjne (24h łącznie). Każdy blok odpowiada jednemu tematowi z programu przedmiotu.

Zajęcia 1–3: Założenia i analiza wymagań (3h)

Zadania:

- Wybór tematu aplikacji i sformowanie zespołu projektowego (1–3 osoby)
- Analiza wymagań funkcjonalnych i нефункциональных
- Wybór stosu technologicznego (backend, PWA, mobile) z uzasadnieniem
- Opracowanie diagramu architektury systemu (backend ↔ API ↔ PWA ↔ mobile)
- Przygotowanie user stories / przypadków użycia
- Stworzenie mockupów UI (np. Figma, Excalidraw, papier)
- Założenie repozytorium Git (monorepo: backend/, pwa/, mobile/)

Efekt końcowy: *Dokument analizy wymagań, diagram architektury, mockupy UI, repozytorium Git*

Zajęcia 4–6: Projektowanie i programowanie PWA (3h)

Zadania:

- Konfiguracja projektu frontendowego (React / Vue / inne)
- Implementacja manifest.json i konfiguracja Service Workera
- Tworzenie głównych widoków i nawigacji (SPA)
- Podłączenie do REST API backendu
- Implementacja trybu offline (cache'owanie zasobów i danych)
- Responsywny design (mobile-first)
- Opcjonalnie: push notifications

Efekt końcowy: *Działająca aplikacja PWA dostępna z przeglądarki, z trybem offline*

Zajęcia 7–9: Projektowanie i programowanie aplikacji mobilnej cross-platform (3h)

Zadania:

- Konfiguracja projektu mobilnego (React Native / Flutter / inne)
- Implementacja nawigacji między ekranami
- Budowa głównych widoków aplikacji
- Integracja z tym samym REST API co PWA
- Obsługa specyficznych funkcji mobilnych (kamera, geolokalizacja, powiadomienia)
- Przechowywanie danych lokalnych (AsyncStorage / SQLite / Hive)
- Build i testowanie na emulatorze lub urządzeniu fizycznym

Efekt końcowy: *Działająca aplikacja mobilna (APK lub build Expo/Flutter)*

Zajęcia 10–12: Wspólny i spójny interfejs graficzny (3h)

Zadania:

- Opracowanie design systemu (paleta kolorów, typografia, komponenty)
- Unifikacja wyglądu PWA i aplikacji mobilnej
- Wspólne komponenty UI (przyciski, karty, formularze, listy)

- Zapewnienie spójnego UX między platformami
- Dostępność (accessibility) – kontrast, rozmiary czcionek, etykiety
- Obsługa trybu jasnego i ciemnego (opcjonalnie)

Efekt końcowy: *Spójny wizualnie interfejs na obu platformach, dokumentacja design systemu*

Zajęcia 13–15: Programowanie backendu i synchronizacja danych (3h)

Zadania:

- Projektowanie modeli danych i bazy danych
- Implementacja REST API (CRUD, filtrowanie, paginacja)
- Mechanizm synchronizacji danych między klientami a serwerem
- Obsługa konfliktów przy edycji offline
- Optymalizacja zapytań i wydajność API
- Opcjonalnie: WebSocket / Server-Sent Events dla real-time updates

Efekt końcowy: *Działający backend z API, synchronizacja danych między PWA i apką mobilną*

Zajęcia 16–18: Zabezpieczanie aplikacji (3h)

Zadania:

- Implementacja systemu uwierzytelniania (JWT / OAuth2 / sesje)
- Rejestracja i logowanie użytkowników
- Autoryzacja – role i uprawnienia (np. admin vs użytkownik)
- Zabezpieczenie API (CORS, rate limiting, walidacja danych wejściowych)
- HTTPS i bezpieczne przechowywanie tokenów
- Ochrona przed typowymi atakami (XSS, CSRF, SQL Injection)
- Bezpieczne przechowywanie danych wrażliwych na urządzeniu mobilnym

Efekt końcowy: *Zabezpieczona aplikacja z działającym systemem logowania i autoryzacji*

Zajęcia 19–21: Testowanie aplikacji (3h)

Zadania:

- Testy jednostkowe backendu (pytest / unittest)
- Testy API (Postman, REST client lub testy automatyczne)
- Testy komponentów frontend/PWA (Jest, React Testing Library)
- Testy aplikacji mobilnej (na emulatorze i/lub urządzeniu)
- Testy integracyjne (komunikacja frontend ↔ backend)
- Testy użyteczności (UX) – np. testy z innymi studentami
- Raport z testów – lista znalezionych i naprawionych błędów

Efekt końcowy: *Raport z testów, zestaw testów w repozytorium*

Zajęcia 22–24: Debugowanie, wdrażanie i prezentacja (3h)

Zadania:

- Debugowanie i naprawa znalezionych błędów
- Wdrożenie backendu (Railway / Render / VPS / Docker)
- Wdrożenie PWA (Vercel / Netlify / GitHub Pages)
- Build aplikacji mobilnej (APK przez Expo / Flutter build)

- Przygotowanie dokumentacji końcowej
- Prezentacja projektu (demo działającej aplikacji, 10–15 minut na zespół)

Efekt końcowy: *Wdrożona aplikacja, dokumentacja końcowa, prezentacja*

Dokumentacja projektu

Do projektu należy dołączyć dokumentację (w repozytorium jako README.md lub osobny plik PDF) zawierającą:

1. **Opis aplikacji** – cel, grupa docelowa, główne funkcjonalności
2. **Architektura systemu** – diagram (backend ↔ API ↔ PWA ↔ mobile), opis komponentów
3. **Wybrana technologia** – uzasadnienie wyboru stosu technologicznego
4. **Opis API** – lista endpointów z opisem (można użyć Swagger/OpenAPI)
5. **Design system** – paleta kolorów, typografia, kluczowe komponenty UI
6. **Opis funkcjonalności** – lista zaimplementowanych feature'ów z krótkim opisem
7. **Zabezpieczenia** – opis zastosowanych mechanizmów bezpieczeństwa
8. **Testowanie** – raport z testów, lista znalezionych i naprawionych błędów
9. **Zrzuty ekranu** – z PWA i aplikacji mobilnej
10. **Instrukcja uruchomienia** – jak uruchomić backend, PWA i apkę mobilną
11. **Napotkane problemy** – opis trudności i sposobów ich rozwiązania
12. **Możliwości rozwoju** – co można by dodać w przyszłości

Kryteria oceny

Kryterium	Punkty
Funkcjonalność PWA (tryb offline, responsywność, Service Worker)	0–15
Funkcjonalność aplikacji mobilnej (natywne funkcje, nawigacja)	0–15
Backend i API (CRUD, synchronizacja, wydajność)	0–15
Spójność interfejsu graficznego (design system, UX)	0–10
Zabezpieczenia (uwierzytelnianie, autoryzacja, bezpieczeństwo API)	0–10
Jakość kodu i architektura (czytelność, wzorce, Git)	0–10
Testowanie (testy, raport z testów)	0–10
Dokumentacja projektu	0–10
Prezentacja i demo	0–5
Razem	100

Skala ocen

- 91–100 pkt – bardzo dobry (5.0)
- 81–90 pkt – dobry plus (4.5)
- 71–80 pkt – dobry (4.0)
- 61–70 pkt – dostateczny plus (3.5)
- 51–60 pkt – dostateczny (3.0)
- 0–50 pkt – niedostateczny (2.0)

Forma oddania projektu

1. **Kod źródłowy** – repozytorium Git (GitHub, GitLab lub inne) ze strukturą: backend/, pwa/, mobile/
2. **Dokumentacja** – README.md lub plik PDF w repozytorium
3. **Działający deploy** – backend i PWA dostępne online (choćby na darmowym hostingu)
4. **Build mobilny** – plik APK lub możliwość uruchomienia przez Expo Go / emulator
5. **Prezentacja** – demonstracja działającej aplikacji na ostatnich zajęciach (10–15 minut na zespół)

Przykładowe tematy aplikacji

Poniżej propozycje tematów – można realizować własny pomysł po konsultacji z prowadzącym:

- **TaskBoard** – aplikacja do zarządzania zadaniami w zespole (w stylu Trello/Kanban)
- **FitTracker** – dziennik treningowy z wykresami postępów i planem ćwiczeń
- **CookBook** – organizer przepisów z listą zakupów i wyszukiwaniem po składnikach
- **EventHub** – aplikacja do organizacji i odkrywania wydarzeń lokalnych
- **BudgetApp** – tracker wydatków z budżetami, kategoriami i wykresami
- **StudyCards** – aplikacja do nauki z fiszkami, quizami i statystykami
- **HabitLoop** – tracker nawyków z seriami, przypomnieniami i motywacją
- **WeatherNow** – aplikacja pogodowa z geolokalizacją, prognozą i mapami
- **ChatBox** – prosty komunikator z wiadomościami w czasie rzeczywistym
- **NoteSync** – aplikacja do notatek z synchronizacją, tagami i wyszukiwaniem

Dodatkowe informacje

- **Praca zespołowa** – projekt można realizować indywidualnie lub w zespołach do 3 osób. Przy większym zespole oczekiwania dotyczące zakresu projektu są wyższe.
- **Konsultacje** – w przypadku problemów technicznych – konsultacje podczas zajęć
- **Git** – zachęcam do korzystania z systemu kontroli wersji od pierwszych zajęć. Historia commitów jest brana pod uwagę przy ocenie.
- **Biblioteki zewnętrzne** – dozwolone jest korzystanie z bibliotek, frameworków i narzędzi open-source
- **AI i generatory kodu** – dozwolone jako wsparcie, ale student musi rozumieć i umieć wyjaśnić każdy fragment kodu
- **Plagiat** – kopiowanie projektu innego zespołu lub całościowe wykorzystanie gotowego projektu z internetu skutkuje oceną niedostateczną